Проект StepPay

ПЛАТФОРМА STEPPAY. СМАРТ-КОНТРАКТЫ STEPESCROW И STEPANCHOR

Описание программы

STEPPAY.00001-01 13 01

Листов 23

2026

СОДЕРЖАНИЕ

1 ОБЩИЕ СВЕДЕНИЯ....3
1.1 Обозначение и наименование программы....3
1.2 Программное обеспечение, необходимое для функционирования....3
1.3 Языки программирования....4
1.4 Нормативные ссылки....4
1.5 Термины и сокращения....4
2 ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ....6
2.1 Классы решаемых задач....6
2.2 Сведения о функциональных ограничениях....6
3 ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ....8
3.1 Состав и структура....8
3.2 Данные и хранение....8
3.3 Жизненный цикл сделки и этапа....9
3.4 Режимы сделок....9
3.5 Подписи, резерв и начало работы....10
3.6 Приемка, голосование и споры....10
3.7 Сроки и аварийный выход....11
3.8 Расчет выплат, комиссия и возвраты....11
3.9 Отказ токена: долг и вывод....12
3.10 Хеш условий и соответствие ядру сделок....12
3.11 Контракт StepAnchor....12
3.12 Инварианты....13
3.13 Перенос сделок приложения на контракт в тестовой сети....13
4 ИСПОЛЬЗУЕМЫЕ ТЕХНИЧЕСКИЕ СРЕДСТВА....15
5 ВЫЗОВ И ЗАГРУЗКА....15
6 ВХОДНЫЕ ДАННЫЕ....16
7 ВЫХОДНЫЕ ДАННЫЕ....16
8 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ....17
8.1 Объект и цель испытаний....17
8.2 Виды и средства испытаний....17
8.3 Результаты....17
9 БЕЗОПАСНОСТЬ, ДОПУЩЕНИЯ И ИЗВЕСТНЫЕ ОГРАНИЧЕНИЯ....18
ПРИЛОЖЕНИЕ А Перечень функций контракта StepEscrow....19
ПРИЛОЖЕНИЕ Б Перечень событий, ошибок и параметров....20
ПРИЛОЖЕНИЕ В Допустимые переходы состояний....21
ПРИЛОЖЕНИЕ Г Соответствие правил ядра сделок и контракта....22

1 ОБЩИЕ СВЕДЕНИЯ

1.1 Обозначение и наименование программы

Наименование: смарт-контракты платформы StepPay (далее контракты): StepEscrow (эскроу по этапам) и StepAnchor (публичный реестр корней якорей истории). Обозначение документа: STEPPAY.00001-01 13 01. Тестовый токен TestRub входит в состав комплекта только для испытаний и в описание функционального назначения не включается.

Платформа StepPay реализует оплату по шагам в цифровых рублях: деньги резервируются и передаются исполнителю только за принятые этапы работы. Контракты являются эталонной реализацией правил на исполняемой среде EVM: те же правила реализованы в ядре сделок платформы (пакет engine), и обе реализации сверяются общими сценариями и случайными прогонами.

1.2 Программное обеспечение, необходимое для функционирования

Для работы контрактов необходима сеть, совместимая с виртуальной машиной Ethereum (EVM), и токен стандарта ERC-20 по интерфейсу transfer, transferFrom и balanceOf. Целевая версия набора инструкций: cancun. Для сборки, испытаний и развертывания используются:

- компилятор Solidity версии 0.8.28 (в проекте подключается пакетом solcjs, загрузка двоичного компилятора не требуется);
- среда разработки Hardhat 2, библиотека ethers версии 6;
- среда выполнения Node.js (версия 20 и выше) и менеджер пакетов pnpm;
- библиотека fast-check для случайных испытаний свойств.

Сторонние библиотеки контрактов (в том числе OpenZeppelin) не используются: интерфейс ERC-20 и защита от повторного входа написаны в самих контрактах.

1.3 Языки программирования

Контракты написаны на языке Solidity 0.8.28. Испытания, сценарии и скрипты развертывания написаны на JavaScript (Node.js), ядро сделок, с которым сверяются контракты, на TypeScript.

1.4 Нормативные ссылки

Документ оформлен с учетом требований Единой системы программной документации: ГОСТ 19.101-77 (виды программных документов), ГОСТ 19.103-77 (обозначение программ и программных документов), ГОСТ 19.104-78 (основные надписи), ГОСТ 19.105-78 (общие требования к программным документам), ГОСТ 19.106-78 (требования к программным документам, выполненным печатным способом), ГОСТ 19.402-78 (описание программы), ГОСТ 19.301-79 (программа и методика испытаний). Структура разделов 1-7 соответствует ГОСТ 19.402-78, раздел 8 построен по ГОСТ 19.301-79 в сокращенной форме.

1.5 Термины и сокращения

Таблица 1 - Термины и сокращения

Термин	Определение
Сделка	Договоренность об оплате работы по этапам, хранящаяся в контракте под номером dealId
Этап	Часть работы с фиксированной суммой, принимаемая и оплачиваемая отдельно
Резерв	Токены, внесенные в контракт и доступные только для выплаты по этапам или возврата
Исполнитель	Сторона, выполняющая работу и получающая оплату (автор проекта в краудфандинге, получатель гранта)
Заказчик	Сторона, вносящая всю сумму и принимающая этапы (в гранте: грантодатель)
Спонсор	Участник краудфандинга, вносящий часть суммы и голосующий за приемку этапов
Приемщик	Независимая сторона, решающая спор между заказчиком и исполнителем
Запасной приемщик	Второй независимый приемщик, который может решить спор со второй половины срока на спор, если основной молчит
Хеш условий (contractHash)	SHA-256 канонической записи условий сделки, хранимый в контракте для сверки с условиями вне цепочки
Якорь	Корень SHA-256 снимка истории платформы, публикуемый в контракте StepAnchor

Термин	Определение
EVM	Виртуальная машина Ethereum
ERC-20	Стандарт интерфейса взаимозаменяемого токена
Базисный пункт (bps)	Одна десятитысячная доля; 100 bps равны одному проценту

2 ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ

2.1 Классы решаемых задач

Контракт StepEscrow решает следующие задачи:

- прием и хранение резервов в токене сделки и учет вкладов каждого участника;
- фиксация подписей сторон и начало работы только после полного резерва и всех подписей;
- последовательное ведение этапов: сдача, приемка, отказ, спор, голосование спонсоров;
- выплата исполнителю за принятые этапы и возврат остатка резерва во всех случаях закрытия сделки;
- пропорциональное деление выплат и возвратов между спонсорами без потери копеек;
- удержание комиссии платформы (до 10 процентов от суммы этапа);
- автоматическую приемку при отсутствии ответа и аварийный выход по срокам, если сторона пропала;
- защиту получателей друг от друга: отказ токена одному адресу не блокирует остальных.

Контракт StepAnchor решает задачу публичной и неизменяемой фиксации корней якорей истории платформы, чтобы переписать историю задним числом без обнаружения было нельзя.

2.2 Сведения о функциональных ограничениях

- Контракт работает с одним токеном, заданным при развертывании. Токены с комиссией при переводе отклоняются, токены с ребейзом и хуками не поддерживаются.

- Контракт не хранит условия сделки (названия этапов, описание, фото), а только суммы этапов, участников, сроки и хеш условий. Сами условия хранятся вне цепочки.

- Подписи сторон в контракте это вызов функции sign с адреса стороны; юридически значимой квалифицированной электронной подписью по 63-ФЗ они не являются.

- Предельные значения: не более 32 этапов в сделке, не более 64 вкладчиков в сделке, комиссия не более 1000 bps.

- Контракт неизменяем: нет владельца, прокси, паузы и обновления. Исправление ошибок требует развертывания нового контракта.

- Независимый аудит контрактов на момент составления документа не проводился.

3 ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

3.1 Состав и структура

Таблица 2 - Состав комплекта контрактов

Файл	Назначение	Строк
contracts/StepEscrow.sol	Эскроу по этапам: три режима, подписи, резерв, голосование, споры, сроки, запасной приемщик, долги и вывод	около 570
contracts/StepAnchor.sol	Публичная запись корней якорей истории платформы по порядку номеров	35
contracts/TestRub.sol	Тестовый токен для испытаний (черный список и комиссия имитируются); в эксплуатацию не входит	около 60
rails/evm-rail.js	Адаптер: контракт как денежный рельс с тем же интерфейсом, что и эмулятор на ядре сделок	
rails/anchor-chain.js, scripts/anchor.js	Публикация корней якорей в сеть и сверка с сетью	
scripts/testnet-deal.js	Прогон сделки в локальной или публичной сети с отчетом о транзакциях	
test/*.test.js	Модульные, сценарные, инвариантные и дифференциальные испытания	

Контракт StepEscrow не наследует других контрактов. Единственная внешняя зависимость это токен ERC-20, адрес которого задается в конструкторе вместе с адресом получателя комиссии. Оба адреса неизменяемы. Конструктор отклоняет нулевые адреса и адрес токена, по которому нет кода контракта.

3.2 Данные и хранение

Состояние хранится в контракте. Основные структуры данных:

Таблица 3 - Структуры данных StepEscrow

Структура	Поля	Назначение
Stage	amount (uint128), status, submittedAt, evidence (bytes32), activeSince, disputedAt	Один этап: сумма, статус, моменты сдачи, начала работы и начала спора, хеш материалов сдачи
Deadlines	fundingSecs, stageSecs, disputeSecs (uint64)	Сроки аварийного выхода в секундах, ноль означает без срока
Params	mode, executor, owner, arbiter, autoAcceptSecs, feeBps, contractHash, deadlines, backupArbiter	Параметры создания сделки (служебная, передается в _create)

Структура	Поля	Назначение
Deal	mode, status, executor, owner, arbiter, backupArbiter, executorSigned, ownerSigned, feeBps, autoAcceptSecs, createdAt, deadlines, contractHash, target, totalReserved, totalContributed, funders[], stages[]	Сделка целиком

Глобальное состояние контракта: dealCount (число сделок), totalLocked (сумма резервов всех сделок), totalOwed (сумма долгов получателям), отображения reserves[dealId][адрес] (текущий резерв участника), contributed[dealId][адрес] (всего внесено участником, вес голоса), votes[dealId][этап][адрес] (голос), owed[адрес] (долг). Веса голосов за и против по этапу накапливаются во внутренних отображениях.

3.3 Жизненный цикл сделки и этапа

Статусы сделки: Funding (сбор и подписи), Active (работа идет), Completed (все этапы приняты), Cancelled (закрыта с возвратом остатка). Статусы этапа: Pending (ждет очереди), Active (в работе), Submitted (сдан), Accepted (принят), Disputed (в споре), Failed (провален). В любой момент в работе, на сдаче или в споре находится не более одного этапа, этапы закрываются строго по порядку. Допустимые переходы приведены в приложении В.

Работа начинается автоматически, когда выполнены три условия: резерв равен целевой сумме, сделку подписали все стороны, статус равен Funding. В этот момент первый этап становится Active и запоминается момент начала работы над ним.

3.4 Режимы сделок

Таблица 4 - Режимы сделок

Режим	Вносит деньги	Принимает этап	Подписывают	Приемщик	При отказе
Own (своя сделка)	заказчик, всю сумму сразу	заказчик	исполнитель и заказчик	обязателен	спор: этап замораживается до решения приемщика
Crowdfunding	любые адреса, кроме автора, до цели	спонсоры голосованием, вес равен внесенной сумме	автор	нет	половина веса и больше против: этап провален, остаток возвращается

Режим	Вносит деньги	Принимает этап	Подписывают	Приемщик	При отказе
Grant (грант)	грантодатель, всю сумму сразу	грантодатель	получатель и грантодатель	нет	этап провален, остаток возвращается грантодателю

Автоприемка (любой адрес после autoAcceptSecs без ответа) действует в режимах Own и Crowdfunding и не действует в гранте.

3.5 Подписи, резерв и начало работы

Функция sign записывает подпись вызывающего адреса, если он исполнитель или (кроме краудфандинга) заказчик, и только пока сделка в статусе Funding. Повторная подпись отклоняется. Функция fund вносит токены: в режимах Own и Grant вносит только заказчик и ровно целевую сумму, в Crowdfunding любой адрес, кроме автора, не более недостающей суммы. Токены списываются через transferFrom, поэтому вкладчик предварительно разрешает контракту списание (approve). После списания контракт сверяет прирост своего баланса с суммой: если пришло не ровно столько (токен с комиссией, ребейз), операция откатывается.

3.6 Приемка, голосование и споры

- submitStage: исполнитель сдает активный этап и прикладывает хеш материалов (описание и фото хранятся вне цепочки).

- approveStage: заказчик (Own) или грантодатель (Grant) принимает сданный этап, выплата проводится сразу.

- rejectStage: отказ с непустой причиной. В режиме Own этап переходит в спор, в гранте этап проваливается и остаток возвращается.

- vote: спонсор голосует за или против, вес равен его вкладу, голос можно менять, пока этап не закрыт. Больше половины суммарного веса за принимает этап, половина и больше против проваливает.

- openDispute: заказчик или исполнитель открывают спор по сданному этапу (режим Own).

- resolveDispute: основной приемщик решает в любое время, запасной только со второй половины срока на спор (не раньше disputedAt плюс половина disputeSecs). Решение pay выплачивает исполнителю, иначе остаток возвращается.

- autoAccept: любой адрес принимает сданный этап, если ответа нет дольше autoAcceptSecs.

- cancel: заказчик (Own, Grant) или автор (Crowdfunding) закрывает сделку и возвращает остаток, если нет сданного или спорного этапа.

3.7 Сроки и аварийный выход

Три необязательных срока задаются при создании сделки (createDealEx, createDealFull) и после их истечения любой адрес может вызвать возврат без согласия сторон:

Таблица 5 - Сроки аварийного выхода

Функция	Условие	Результат
expireFunding	сделка в статусе Funding, срок fundingSecs от создания истек	сделка отменена, все резервы возвращены
expireStage	этап Active, срок stageSecs от начала работы над этапом истек	этап провален, остаток возвращен
expireDispute	этап Disputed, срок disputeSecs от открытия спора истек	этап провален, остаток возвращен заказчику

Срок спора допустим только в режиме Own. Если срок равен нулю, соответствующий выход недоступен. Время берется из block.timestamp, сроки измеряются сутками, поэтому погрешность в секунды не влияет на результат.

3.8 Расчет выплат, комиссия и возвраты

Сумма этапа списывается с резервов вкладчиков пропорционально их текущим резервам методом наибольших остатков: каждому достается целая часть доли, а недостающие копейки по одной отдаются тем, у кого остаток от деления больше. Сумма частей всегда равна сумме этапа, отклонение от точной доли не больше одной копейки. Исполнитель получает сумму этапа за вычетом комиссии (amount умножить на feeBps и разделить на 10000,

округление вниз), комиссия переводится получателю комиссии. При закрытии сделки с провалом или отменой остаток резерва возвращается вкладчикам в тех же долях, в каких он у них остался.

3.9 Отказ токена: долг и вывод

Выплата и возврат выполняются циклом по вкладчикам. Чтобы один получатель, которому токен отказал в приеме (черный список, запрет получения), не блокировал остальных, внутренний перевод _push при неудаче не откатывает операцию, а записывает сумму получателю в долг: owed[адрес] и totalOwed увеличиваются, выпускается событие Owed. Получатель забирает долг сам функцией claim(to), при необходимости на другой адрес. Если перевод при claim неудачен, операция откатывается и долг сохраняется. Токены долга остаются на балансе контракта, поэтому инвариант баланса имеет вид: баланс токенов контракта равен totalLocked плюс totalOwed.

3.10 Хеш условий и соответствие ядру сделок

При создании сделки передается contractHash: SHA-256 канонической записи условий (ключи по алфавиту, без пробелов), которую строит ядро сделок. Контракт хранит хеш и выводит его в событии DealCreated, поэтому условия вне цепочки можно сверить с цепочкой пересчетом хеша. Контракт хеш не проверяет и не интерпретирует. Правила, перечисленные в приложении Г, реализованы и в ядре сделок, и в контракте, а их эквивалентность проверяется испытаниями (раздел 8).

3.11 Контракт StepAnchor

Контракт принимает от издателя (адрес задается в конструкторе и неизменяем) корень якоря функцией anchor(n, root). Номера идут строго подряд, начиная с единицы, нулевой корень запрещен, перезапись записанного корня невозможна. Для каждого номера хранятся корень (roots) и время записи (anchoredAt), выпускается событие Anchored. Контракт ничего не знает о сделках и деньгах. Корень якоря вычисляется платформой

как SHA-256 канонической записи снимка: длина журнала и хеш последней записи для каждой сделки, длина и хеш журнала операций с деньгами, корень предыдущего якоря.

3.12 Инварианты

Следующие свойства проверяются после каждого действия в случайных прогонах:

- баланс токенов контракта равен totalLocked плюс totalOwed;
- totalLocked равен сумме резервов по всем сделкам, а резервы каждой сделки совпадают с totalReserved внутри нее;
- завершенная или отмененная сделка не держит резервов;
- в активной сделке резерв равен сумме непринятых этапов;
- в сделке на этапе сбора резерв не превышает цель;
- токены не создаются и не пропадают: сумма балансов участников и контракта постоянна;
- этапы идут по порядку: принятые, не больше одного в работе, остальные ждут.

3.13 Перенос сделок приложения на контракт в тестовой сети

Серверная часть приложения содержит зеркало: компонент, который после каждого действия по сделке воспроизводит его на контракте StepEscrow. Источником истины остается ядро сделок, зеркало читает журнал сделки и по порядку отправляет в сеть соответствующие вызовы: sign, fund, submitStage, approveStage, rejectStage, openDispute, vote, resolveDispute, autoAccept, expireFunding, expireStage, expireDispute, cancel. Зеркало включается переменными окружения (адрес узла, адреса контрактов, ключ счета платформы). Если они не заданы или заданы неверно, приложение работает как прежде.

У каждого участника есть собственный счет в сети. Закрытый ключ счета выводится функцией HKDF из секрета платформы и идентификатора участника, поэтому в базе данных ключи не хранятся. Газ оплачивает счет платформы. Токен tRUB тестовый: его выпуск открыт, ценности он не имеет и цифровым рублем не является. При включенном зеркале пополнение кошелька приложения сначала выпускает токены на счет участника, затем зачисляет сумму в ядре, а баланс «Свободно» сверяется с балансом токенов на счете.

Действия, привязанные ко времени (автоприемка, выход по сроку), повторяются каждые 30 секунд до 20 раз, пока контракт не примет вызов. Сбой сети помечает сделку как не перенесенную и не останавливает приложение. Запрос GET /deals/:id/chain/verify сравнивает статусы этапов и резервы в приложении и на контракте и сообщает расхождения. Ограничение: транзакции от имени участника отправляет платформа, а не сам участник со своего кошелька; в производственной версии подпись должна выполняться ключом участника.

4 ИСПОЛЬЗУЕМЫЕ ТЕХНИЧЕСКИЕ СРЕДСТВА

Контракты исполняются в любой сети, совместимой с EVM и поддерживающей набор инструкций cancun, на узлах этой сети. Для разработки и испытаний требуется компьютер с процессором архитектуры x86-64 или ARM64, не менее 4 ГБ оперативной памяти, Node.js версии 20 и выше, доступ к хранилищу пакетов (для первичной установки зависимостей). Для работы в публичной тестовой сети необходим адрес удаленного вызова процедур (RPC) узла сети, счет с тестовыми монетами для оплаты газа и закрытый ключ этого счета, который передается только через переменные окружения.

5 ВЫЗОВ И ЗАГРУЗКА

Развертывание выполняется скриптом scripts/testnet-deal.js (сделка целиком) и scripts/anchor.js (контракт StepAnchor и публикация якорей). Порядок развертывания StepEscrow:

- развернуть токен ERC-20 сделок (для испытаний TestRub);
- развернуть StepEscrow с параметрами (адрес токена, адрес получателя комиссии);
- для каждой сделки вызвать createDeal, createDealEx или createDealFull с условиями; участники разрешают контракту списание токена (approve) и вызывают sign и fund;
- дальнейшие вызовы: submitStage, approveStage, rejectStage, vote, openDispute, resolveDispute, autoAccept, cancel, expireFunding, expireStage, expireDispute, claim.

Параметры публичной сети задаются переменными окружения: SEPOLIA_RPC_URL или BASE_SEPOLIA_RPC_URL, DEPLOYER_PRIVATE_KEY (только тестовый ключ), EXPLORER_URL. Закрытые ключи в репозитории и журналах не хранятся. Команды: pnpm

chain:local (локально), pnpm chain:sepolia (публичная тестовая сеть), pnpm anchor:publish (публикация якорей).

6 ВХОДНЫЕ ДАННЫЕ

Таблица 6 - Входные данные создания сделки

Параметр	Тип	Описание и ограничения
mode	Mode	Own (0), Crowdfunding (1), Grant (2)
executor	address	Исполнитель, не нулевой адрес
owner	address	Заказчик или грантодатель; для Crowdfunding нулевой адрес; не равен исполнителю
arbiter	address	Приемщик: обязателен в Own, нулевой в остальных режимах; не совпадает со сторонами
stageAmounts	uint128[]	Суммы этапов в наименьших единицах токена, от 1 до 32 элементов, каждая больше нуля
autoAcceptSecs	uint64	Срок автоприемки в секундах; для режимов Own и Crowdfunding больше нуля, для Grant любое
feeBps	uint16	Комиссия в базисных пунктах, не более 1000
contractHash	bytes32	SHA-256 условий сделки
deadlines	Deadlines	Сроки в секундах (createDealEx, createDealFull); срок спора только для Own
backupArbiter	address	Запасной приемщик (createDealFull): только Own, нужен срок на спор, не совпадает с исполнителем, заказчиком и основным приемщиком

Прочие входные данные: dealId и номер этапа (нумерация с нуля), хеш материалов сдачи (evidence), хеш причины отказа или спора (reason, не нулевой), сумма взноса, голос, решение приемщика. Для StepAnchor: номер якоря и его корень.

7 ВЫХОДНЫЕ ДАННЫЕ

Результатом работы контрактов являются изменения состояния, перевод токенов и события журнала транзакций. Перечень событий приведен в приложении Б. Для чтения состояния предусмотрены функции: getDeal (режим, статус, участники, хеш условий, цель, резерв, число этапов), getStage (сумма, статус, момент сдачи, хеш материалов), getStageTimes, getDeadlines, backupArbiterOf, fundersOf, reservedSum, isSigned, а также открытые поля token, feeRecipient, dealCount, totalLocked, totalOwed, reserves, contributed, votes, owed. Контракт StepAnchor отдает count, roots, anchoredAt и publisher.

8 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

8.1 Объект и цель испытаний

Объект испытаний: контракты StepEscrow и StepAnchor (раздел 3). Цель: подтвердить, что контракты реализуют правила раздела 3, сохраняют инварианты раздела 3.12 при любых последовательностях вызовов, дают те же результаты, что эталонное ядро сделок, и не допускают потери и создания средств.

8.2 Виды и средства испытаний

Таблица 7 - Виды испытаний

Вид	Содержание	Команда запуска
Модульные	Права вызова, переходы статусов, подписи, голосование, споры, сроки, запасной приемщик, долг и вывод, комиссия, предельное число вкладчиков, отказ токена и токен с комиссией	pnpm --filter @steppay/contracts test
Сценарные (рельсы)	Одни и те же 16 сценариев исполняются на ядре сделок и на контракте, результаты (принятые и отклоненные шаги, балансы, статусы) сравниваются до копейки	входит в test
Инвариантные	Случайные последовательности действий в трех режимах, проверка инвариантов раздела 3.12 после каждого действия	pnpm --filter @steppay/contracts test:invariants
Дифференциальные	Случайные сценарии (режимы, неровные взносы, комиссия, споры, автоприемка, сроки, запасной приемщик, шум из неверных вызовов) на ядре и контракте, расхождения недопустимы	FC_DIFF_RUNS=500 pnpm --filter @steppay/contracts test:differential
Свойства ядра	400 хаотичных и 80 направляемых случайных прогонов ядра сделок, свыше двадцати условий после каждого шага	pnpm --filter @steppay/engine test:properties
Проверка испытаний мутациями	В контракт вносятся осознанные поломки (сдвиг границ сроков и половины срока на спор), испытания обязаны упасть	ручной прогон дифференциальных испытаний
Газ	Измерение стоимости приемки этапа и возврата при предельных 64 вкладчиках	входит в test

8.3 Результаты

На момент составления документа: контракты StepEscrow и StepAnchor проходят 48 испытаний пакета контрактов, 72 испытания ядра сделок, дифференциальные прогоны на 200 сценариях без расхождений, инвариантные прогоны без нарушений. Поломка границ сроков и половины срока на спор в контракте обнаруживается дифференциальными прогонами (поломка половины срока обнаруживается на 700 сценариях, на меньшем

числе не гарантируется). При 64 вкладчиках приемка этапа с выплатой стоит около 1,27 млн единиц газа, провал этапа с возвратом всем около 1,22 млн единиц газа, что существенно ниже лимита блока. Испытания выполнены в локальной сети Hardhat. Контракты развернуты в публичной тестовой сети Sepolia, перенос сделок приложения проверен автоматическими тестами с подделкой сети и отдельными тестами на локальной сети; ручная проверка в Sepolia продолжается, ее результаты в состав данного документа не включены.

9 БЕЗОПАСНОСТЬ, ДОПУЩЕНИЯ И ИЗВЕСТНЫЕ ОГРАНИЧЕНИЯ

Допущения, на которые опирается контракт, и меры защиты:

Таблица 8 - Допущения и меры защиты

Вопрос	Решение
Повторный вход	Функции с вызовами токена защищены собственным запретом повторного входа (nonReentrant); функции без внешних вызовов (sign, submitStage, openDispute) без защиты
Тип токена	Один токен на контракт, задается при развертывании; токены с комиссией при переводе отклоняются сверкой прироста баланса; ребейз и хуки не поддерживаются
Блокировка выплат токеном	Отказ токена одному получателю превращается в долг с выводом через claim, остальные получают деньги
Администрирование	Привилегированных адресов нет, кроме неизменяемого получателя комиссии и издателя StepAnchor; пауза и обновление отсутствуют
Время	block.timestamp; сроки сутки и больше
Переполнение	Solidity 0.8 с проверкой арифметики; суммы этапов хранятся в uint128, суммы по сделке в uint256
Спам созданием сделок	Создание доступно любому адресу, но средства не двигаются и вкладчики добровольно вносят деньги
Подкуп приемщика	Не технический риск; снижается реестром приемщиков платформы, репутацией и запасным приемщиком, но не исключается

Известные ограничения и непроверенное: независимый аудит не проводился (подготовлено задание для аудитора), формальной верификации нет, запуск в публичной сети не выполнялся, поведение с реальным токеном цифрового рубля зависит от его интерфейса и должно быть проверено отдельно. Допущения и открытые вопросы для аудитора приведены в документе docs/audit-brief.md репозитория проекта.

ПРИЛОЖЕНИЕ А

(справочное)

Перечень функций контракта StepEscrow

Таблица 9 - Функции StepEscrow

Функция	Кто вызывает	Условие и действие
createDeal(mode, executor, owner, arbiter, stageAmounts, autoAcceptSecs, feeBps, contractHash)	любой адрес	Создает сделку без сроков. Возвращает dealId
createDealEx(..., deadlines)	любой адрес	То же со сроками аварийного выхода
createDealFull(..., deadlines, backupArbiter)	любой адрес	То же и с запасным приемщиком
sign(dealId)	исполнитель, заказчик (кроме Crowdfunding)	Статус Funding; повторная подпись отклоняется; может запустить работу
fund(dealId, amount)	заказчик (Own, Grant: ровно цель); любой, кроме автора (Crowdfunding)	Статус Funding, нужен approve токена; сверка прироста баланса; может запустить работу
submitStage(dealId, stage, evidence)	исполнитель	Этап Active, сделка Active; запоминает момент сдачи
approveStage(dealId, stage)	заказчик или грантодатель	Этап Submitted (кроме Crowdfunding); выплата
rejectStage(dealId, stage, reason)	заказчик или грантодатель	Этап Submitted, причина не нулевая; Own: спор, Grant: провал и возврат
vote(dealId, stage, accept)	вкладчик (Crowdfunding)	Этап Submitted; вес равен вкладу; перевес принимает, половина и больше против проваливает
openDispute(dealId, stage, reason)	заказчик или исполнитель (Own)	Этап Submitted; переводит этап в спор, запоминает момент
resolveDispute(dealId, stage, pay)	основной приемщик; запасной со второй половины срока спора	Этап Disputed; pay выплачивает, иначе возврат
autoAccept(dealId, stage)	любой адрес	Этап Submitted, Own или Crowdfunding, срок autoAcceptSecs от сдачи истек; выплата
cancel(dealId)	заказчик (Own, Grant); автор (Crowdfunding)	Нет этапов Submitted и Disputed; возврат остатка
expireFunding(dealId)	любой адрес	Статус Funding, срок сбора истек; отмена и возврат
expireStage(dealId, stage)	любой адрес	Этап Active, срок этапа истек; провал и возврат
expireDispute(dealId, stage)	любой адрес	Этап Disputed, срок спора истек; провал и возврат заказчику
claim(to)	адрес с долгом	owed больше нуля; переводит долг на адрес to; при неудаче перевода откат
getDeal, getStage, getStageTimes, getDeadlines, backupArbiterOf, fundersOf, reservedSum, isSigned	любой адрес	Чтение состояния

ПРИЛОЖЕНИЕ Б

(справочное)

Перечень событий, ошибок и параметров

Таблица 10 - События StepEscrow

Событие	Когда выпускается
DealCreated(dealId, mode, executor, owner, arbiter, contractHash, target)	Создана сделка
Signed(dealId, who)	Сторона подписала
Funded(dealId, from, amount, totalReserved)	Внесен резерв
Activated(dealId)	Началась работа
StageSubmitted(dealId, stage, evidence)	Этап сдан
StageAccepted(dealId, stage, closedBy)	Этап принят (owner, votes, auto, arbiter)
StageFailed(dealId, stage, closedBy)	Этап провален (owner, votes, arbiter, expired)
StageRejected(dealId, stage, reason)	Отказ по этапу
Voted(dealId, stage, voter, accept)	Голос спонсора
DisputeOpened(dealId, stage, by, reason)	Открыт спор
Payout(dealId, stage, executor, amount, fee)	Выплата исполнителю и комиссия
Refund(dealId, to, amount)	Возврат остатка вкладчику
Completed(dealId)	Все этапы приняты
Cancelled(dealId, by)	Сделка отменена
FundingExpired(dealId)	Закрыта по сроку сбора
Owed(to, amount)	Перевод не удался, записан долг
Claimed(from, to, amount)	Долг выведен

Таблица 11 - Ошибки и предельные параметры

Имя	Значение и смысл
Unauthorized	Вызывающий не имеет права на действие
BadState	Действие недопустимо в текущем состоянии
BadInput	Неверные входные данные
MAX_STAGES	32 этапа в сделке
MAX_FUNDERS	64 вкладчика в сделке
MAX_FEE_BPS	1000 базисных пунктов (10 процентов)
StepAnchor: Anchored(n, root, at)	Записан корень якоря номер n
StepAnchor: Unauthorized, BadNumber	Не издатель; номер не следующий или пустой корень

ПРИЛОЖЕНИЕ В

(справочное)

Допустимые переходы состояний

Таблица 12 - Переходы статусов сделки

Из	В	Причина
Funding	Active	Резерв равен цели и все подписали
Funding	Cancelled	cancel или expireFunding
Active	Completed	Принят последний этап
Active	Cancelled	Провален этап, отмена заказчиком или автором, expireStage, expireDispute

Таблица 13 - Переходы статусов этапа

Из	В	Причина
Pending	Active	Начало работы или приемка предыдущего этапа
Active	Submitted	submitStage
Active	Failed	expireStage
Submitted	Accepted	approveStage, перевес голосов, autoAccept
Submitted	Failed	rejectStage (Grant), половина голосов против
Submitted	Disputed	rejectStage (Own), openDispute
Disputed	Accepted	resolveDispute: pay
Disputed	Failed	resolveDispute: возврат, expireDispute

ПРИЛОЖЕНИЕ Г

(справочное)

Соответствие правил ядра сделок и контракта

Таблица 14 - Соответствие ядра и контракта

Правило	Ядро сделок (TypeScript)	Контракт (Solidity)
Режимы	own, crowdfunding, grant	Mode.Own, Mode.Crowdfunding, Mode.Grant
Хеш условий	contractHash от канонической записи	contractHash в сделке и событии
Подписи	signContract	sign
Резерв	fund	fund
Сдача, приемка, отказ	submitStage, approveStage, rejectStage	submitStage, approveStage, rejectStage
Голосование	voteStage	vote
Споры	openDispute, replyDispute, resolveDispute	openDispute, resolveDispute (переписка вне цепочки)
Автоприемка и часы	tick	autoAccept, expire*
Сроки	expireFunding, expireStage, expireDispute	expireFunding, expireStage, expireDispute
Запасной приемщик	backupArbiterId в resolveDispute	createDealFull, resolveDispute
Деление выплат	splitProRata	_split
Отмена	cancelDeal	cancel

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Изм.	Номера листов (страниц) измененных / замененных / новых / аннулированных	Всего листов (страниц) в докум.	№ докум.	Входящий № сопроводит. докум. и дата	Подп.	Дата